

DSA IN JAVA

One-Month Study Companion

30-day course plan + 29 topic-focused problem walkthroughs

Built for active practice in Java: try the problem first, compare brute-force / improved / best patterns, then re-code the best approach from memory.

Includes the course outline schedule, linked video resources, LeetCode matches, and the problems from your messages and screenshots.

Screen-first layout | Java solutions | September 2026

How to use the month

Treat “get acquainted” as: attempt each prompt, identify its pattern, write or trace a correct solution, and state time and space complexity. This is a fast first pass; revisit misses rather than trying to memorize code.

Daily practice rhythm

- 20 min — recall yesterday’s patterns without notes.
- 35–45 min — watch only the relevant video section; write a few short notes.
- 2.5–3 hours — solve the scheduled problems in Java. Give each problem an honest attempt before looking at a hint or editorial.
- 20 min — record the pattern, complexity, bug, and a retry date. Re-solve misses the next day and again 3–4 days later.

The supplied course plan allocates the first 21 days mainly to ST-1 and uses the remaining days for a first pass over ST-2. Contest slots without actual prompts are reserved for unseen problems matching the listed topic.

Day-by-day course schedule

Day 01 Unit 1: Big-O; time vs space; analyze loops and a recursive function. Implement prime factorization and Euclidean GCD. LeetCode practice: [Distinct Prime Factors of Product of Array](#) and [Find Greatest Common Divisor of Array](#).

Day 02 Finish Unit 1: distribute M items around N positions from K using the wraparound formula ([GFG problem](#)); compare brute-force and improved Two Sum ([Two Sum](#)); describe one randomized algorithm and solve [Shuffle an Array](#); implement a divide-and-conquer sort ([Sort an Array](#)).

Day 03 Unit 2: prefix/suffix sums and difference arrays; two pointers. Solve pair sum K with a sorted array ([Two Sum II](#)). Begin Practice Contest 2: 1 of its 3 medium array problems.

Day 04 Fixed-size sliding window; solve maximum sum of a size-k window (LeetCode equivalent: [Maximum Average Subarray I](#)). Finish Contest 2’s other 2 array problems.

Day 05 In-place reversal, array rotation, partitioning. Solve [Rotate Array](#). Begin Practice Contest 3: one mixed array/string problem.

Day 06 String reverse, palindrome, anagram, compression, substrings. Solve [Valid Anagram](#), then finish Contest 3’s other 2 mixed problems.

Day 07 Binary search; KMP prefix-function basics; Z-function; string matching and rolling-hash idea. Implement one search template; try [Find the Index of the First Occurrence in a String](#) and [Repeated DNA Sequences](#).

Day 08 1D DP on arrays: solve LIS ([Longest Increasing Subsequence](#)) and LCS ([Longest Common Subsequence](#)). Write the state, transition, base case, and complexity for each.

Day 09 Unit 3: greedy choice, proof intuition, exchange argument, interval scheduling/activity selection. Solve [Non-overlapping Intervals](#) and [Minimum Number of Arrows to Burst Balloons](#).

Day 10 Fractional knapsack, coin/fraction and min/max greedy examples. Implement fractional knapsack from a small custom input; compare its greedy choice with a case where greedy coin-change fails.

Day 11 Job scheduling with deadlines/profit, meeting/task scheduling, greedy with sorting and priority queues. Implement the course’s job-sequencing exercise; use [Meeting Rooms II](#) for interval/heap practice. Complete Practice Contest 4: 3 greedy problems.

Day 12 Greedy on strings (lexicographic/reordering) and greedy + DP hybrids. Complete Practice Contest 5: 3 mixed greedy/array/string problems. Re-solve the hardest miss from Days 9–11.

Day 13 Unit 4: implement stack operations using an array; balanced parentheses ([Valid Parentheses](#)); expression evaluation ([Evaluate Reverse Polish Notation](#)).

Day 14 Monotonic stack and next-greater-element pattern ([Next Greater Element I](#)). Complete Practice Contest 6: 3 stack/queue problems; for each, identify whether a stack, queue, or deque fits.

Day 15 Queue, circular queue, deque; implement the circular queue ([Design Circular Queue](#)); solve sliding-window maximum ([Sliding Window Maximum](#)). Review stack + greedy/DP combinations.

Day 16 Unit 5: Java HashMap/HashSet, frequency counting, collisions (chaining vs open addressing). Attempt “Noise in the Library” from the instructor’s statement; do a HashMap frequency-count voting solution (practice: [Majority Element](#)). Begin Contest 7: 1 hashing problem.

Day 17 Two Sum variants; “balance a scale with given weights” (use a HashSet of reachable sums; LeetCode practice: [Partition Equal Subset Sum](#)); solve [Subarray Sum Equals K](#). Contest 7: problem 2.

Day 18 Subarray constraints, coordinate compression, string hashing/rolling hash and substring search. Use [Longest Duplicate Substring](#) to inspect rolling-hash tradeoffs. Contest 7: problem 3.

Day 19 Map-based DP; solve [Word Break](#) and one mixed hashing problem. Revisit the most error-prone concepts from Units 1–5.

Day 20 ST-1 review: make a one-page pattern sheet for complexity, arrays/strings, greedy, stack/queue, hashing. Do a 90-minute closed-notes mixed set drawn from the problems already attempted.

Day 21 ST-1 mock and repair: take a second timed set; review every miss and re-code at least 3 from a blank editor. Prioritize this day if the exam falls before Day 30.

Day 22 Unit 6: recursion trees, base cases, DFS; implement Tower of Hanoi and generating binary strings of length n . Start Practice Contest 8: 1 backtracking problem.

Day 23 Backtracking: subsets ([Subsets](#)), string permutations ([Permutations](#)), combinations ([Combinations](#)), robot/grid paths ([Unique Paths](#)). Finish Contest 8’s other 2 problems.

Day 24 Backtracking with pruning: Sudoku ([Sudoku Solver](#)), N-Queens ([N-Queens](#)), and Rat in a Maze/all paths (custom grid DFS; related: [Unique Paths III](#)). Contest 9: 3 mixed recursion/backtracking problems; at minimum attempt each and fully code one.

Day 25 DP applications: 0/1 knapsack (practice equivalent: [Partition Equal Subset Sum](#)); count ways using steps 1/2/3 (related: [N-th Tribonacci Number](#)); Matrix Chain Multiplication (custom interval-DP table; related: [Minimum Score Triangulation of Polygon](#)).

Day 26 Unit 7: implement preorder/inorder/postorder ([Binary Tree Preorder Traversal](#), [Inorder](#), [Postorder](#)); height ([Maximum Depth of Binary Tree](#)); BST LCA ([Lowest Common Ancestor of a Binary Search Tree](#)); kth smallest ([Kth Smallest Element in a BST](#)). Contest 10: 3 tree problems; use 1 as a fully coded timed problem.

Day 27 Trees/BST/trie: search/insert/delete, validation, range sum, floor/ceil; diameter and path sum; max-independent-set idea; trie insert/search/prefix ([Implement Trie](#)). Contest 11: 3 mixed tree/DP problems; attempt all, code one fully, explain the other two.

Day 28 Unit 8: min/max heap, Java PriorityQueue, stream min/max (practice: [Kth Largest Element in a Stream](#)); implement heap sort; write a checker for the max-heap property.

Day 29 Unit 9: adjacency list, BFS/DFS, connected components, cycle detection, bipartite check; minimum-edge path by BFS; directed path ([All Paths From Source to Target](#)); number of islands ([Number of Islands](#)).

Day 30 Shortest paths and graph algorithms: unweighted shortest path, Dijkstra ([Network Delay Time](#)), Bellman–Ford (custom graph with a negative edge), DSU, Kruskal/Prim (related: [Min Cost to Connect All Points](#)), topological sort ([Course Schedule](#)), DAG path/counting DP, and shortest-path DP. Finish by re-solving 2 ST-1 misses.

Video links from the course plan

- [Kunal Kushwaha — Java + DSA course playlist](#) — Java implementation reference; his official syllabus includes arrays, search/sort, complexity, recursion, linked data structures, trees, hashing, DP, heaps, and graphs.
- [Abdul Bari — Time complexity / algorithm analysis](#)
- [freeCodeCamp — Data Structures and Algorithms full course](#)
- [Abdul Bari — Greedy method](#)
- [freeCodeCamp — Stack data structure](#)
- [Abdul Bari — Hashing](#)

- [Recursion and backtracking](#)
- [William Fiset — Binary trees and BST](#)
- [Abdul Bari — Heap data structure](#)
- [William Fiset — Graph theory](#)
- [Striver A2Z DSA playlist](#) — optional topic-wise explanations and problem walk-throughs.
- [LeetCode Study Plans](#) — optional topic practice lists.

Where the 29 workbook chapters fit

- Day 1 — Chapters 1–3: prime factors, GCD, distribute items around a circle.
- Day 4 — Chapters 4–5: first negative and distinct values in every fixed-size window.
- Day 5 — Chapters 11–12: array rotation and clockwise matrix-ring rotation.
- Day 6 — Chapters 6–8: distinct anagram substrings, permutation in a string, and subarray sum.
- Day 7 — Chapters 9–10: pattern search and sorted-array two pointers.
- Days 8–9 — Chapters 13–16: Fibonacci, 0/1 knapsack, DP primer, and minimum Fibonacci terms.
- Days 10–12 — Chapters 17–20: coin change, platforms, remove-k-digits, and lexicographic subsequence.
- Day 13 — Chapters 22–23 and 25: balanced brackets, expression evaluation, and array stack.
- Day 14 — Chapters 24 and 26–29: next-greater element and linked-list/array/circular queues plus reversing first k.
- Days 29–30 — Chapter 21: Water Connection Problem during graph review.

For each assigned workbook chapter, make one honest attempt and code the best approach. Read the brute-force and improved versions to understand the progression; re-code a second version only when comparing the pattern is useful.

All code examples are Java method/class snippets. Assume `import java.util.*;` where needed; helper methods are shown with the solution that uses them.

Practice loop for each workbook problem

1) Cover the solution and spend 20–25 minutes on your own. 2) Write a brute-force version first. 3) Name the pattern and state its invariant. 4) Study the improved and best version. 5) Close the page and re-code the best version in Java. 6) Save one test case that used to confuse you.

Read this before the problem set

- The first-negative code was fenced as Python, but its contents are Java. The workbook labels it as Java and preserves its “0 when no negative exists” behavior.
- The GFG page is “count distinct elements in every window.” The pasted `countAnagram(text, word)` method instead counts distinct fixed-length substrings that are anagrams of word. Both problems are included separately.
- The statement “s2 contains a permutation of s1” is a third, separate boolean problem. It is included as a LeetCode 567-style task.
- “Distribute M items in a circle” means consecutive distribution with wraparound. It is not Josephus elimination; LeetCode 1823 is listed only as a related cyclic problem.
- The screenshot’s lexicographic subsequence constraint is “each included character occurs at least k times.” The workbook keeps that wording and the lowercase alphabet assumption visible.

Each problem has an original short statement for practice, three solution levels in Java, the core pattern, complexity, tips, and clickable source / LeetCode links where available.

Java DSA problem workbook

29 problem and pattern chapters. Each chapter moves from a simple baseline to a more efficient solution; the best method depends on the constraints in the actual prompt.

01. Prime factorization

Problem: Given a positive integer n , return its prime factors with multiplicity. For example, $84 = 2 \times 2 \times 3 \times 7$.

Pattern: Factor stripping; divisors only need to be tested through $\sqrt{\text{remaining } n}$.

1. Brute force: try every possible divisor

$O(n)$ time, $O(\log n)$ output space

Test every integer from 2 upward and divide it out whenever it is a factor.

```
static List<Long> factorsBrute(long n) {
    List<Long> out = new ArrayList<>();
    for (long d = 2; d <= n; d++) {
        if (n % d == 0) {
            out.add(d);
            while (n % d == 0) n /= d;
        }
    }
    return out;
}
```

2. Better: divide by 2, then odd divisors

$O(\sqrt{n})$ time, $O(\log n)$ output space

After 2, only odd candidates are needed. Once $d \cdot d$ exceeds the remaining n , that remainder is prime.

```
static List<Long> factorsSqrt(long n) {
    List<Long> out = new ArrayList<>();
    for (long d = 2; d <= n / d; d = (d == 2 ? 3 : d + 2)) {
        while (n % d == 0) { out.add(d); n /= d; }
    }
    if (n > 1) out.add(n);
    return out;
}
```

3. Best for many bounded queries: smallest-prime-factor table

Precompute $O(M \log \log M)$; each query $O(\log n)$

Build SPF once up to the largest allowed value M , then repeatedly divide by $\text{spf}[n]$. For one very large n , the sqrt method is simpler.

```
static int[] buildSpf(int max) {
    int[] spf = new int[max + 1];
    for (int i = 2; i <= max; i++) if (spf[i] == 0) {
        spf[i] = i;
        if ((long)i * i <= max)
            for (int j = i * i; j <= max; j += i)
                if (spf[j] == 0) spf[j] = i;
    }
    return spf;
}

static List<Integer> factorsWithSpf(int n, int[] spf) {
    List<Integer> out = new ArrayList<>();
    while (n > 1) { out.add(spf[n]); n /= spf[n]; }
    return out;
}
```

Tips and edge cases

Use $d \leq n / d$ instead of $d \cdot d \leq n$ to avoid multiplication overflow. Decide whether the output should repeat a prime ($84 \rightarrow 2, 2, 3, 7$) or list each prime once. The SPF method is for many queries in a known range.

GeeksforGeeks: [Open source problem / topic](#)

LeetCode matches or related practice: [Distinct Prime Factors Of Product Of Array](#)

02. GCD of two numbers

Problem: Given nonnegative integers a and b , return their greatest common divisor. Use $\text{gcd}(0, x) = x$.

Pattern: Divisibility; Euclidean remainder invariant $\text{gcd}(a, b) = \text{gcd}(b, a \bmod b)$.

1. Brute force: test downward from $\min(a, b)$

$O(\min(a, b))$ time, $O(1)$ space

The first number that divides both is the answer.

```
static long gcdBrute(long a, long b) {
    long limit = Math.min(a, b);
    for (long d = limit; d >= 1; d--)
        if (a % d == 0 && b % d == 0) return d;
    return 0;
}
```

2. Better: repeated subtraction

$O(\max(a, b))$ worst-case time, $O(1)$ space

Replace the larger value by the difference until the values meet. It is correct but slow for uneven magnitudes.

```
static long gcdSubtract(long a, long b) {
    if (a == 0) return b;
    if (b == 0) return a;
    while (a != b) {
        if (a > b) a -= b;
        else b -= a;
    }
    return a;
}
```

3. Best: Euclidean algorithm

$O(\log \min(a, b))$ time, $O(1)$ space

Remainders shrink quickly. This is the standard implementation.

```
static long gcd(long a, long b) {
    a = Math.abs(a); b = Math.abs(b);
    while (b != 0) {
        long r = a % b;
        a = b;
        b = r;
    }
    return a;
}
```

Tips and edge cases

Normalize negative inputs if they are allowed. To compute lcm safely, use $(a / \text{gcd}(a, b)) * b$ rather than $a * b / \text{gcd}$.

GeeksforGeeks: [Open source problem / topic](#)

LeetCode matches or related practice: [Find Greatest Common Divisor Of Array](#)

03. Distribute M items in a circle of N positions

Problem: There are N positions numbered 1 to N. Distribute M items to consecutive positions, starting at position K and wrapping from N back to 1. Return the position that receives the last item. Example: N=5, M=8, K=2 gives position 4.

Pattern: Circular indexing; convert to zero-based, add M - 1 steps, take modulo N, then convert back to one-based.

1. Brute force: advance one item at a time

O(M) time, O(1) space

Move one position per item and wrap around when reaching N.

```
static int distributeBrute(int n, int m, int k) {
    int pos = k;
    for (int item = 1; item < m; item++) {
        pos++;
        if (pos > n) pos = 1;
    }
    return pos;
}
```

2. Better: simulate with a wrap condition

O(M) time, O(1) space

Use zero-based indexing and keep the position modulo n after each step.

```
static int distributeLoop(int n, int m, int k) {
    if (n <= 0 || m <= 0 || k < 1 || k > n) throw new IllegalArgumentException();
    int pos = k - 1;
    for (int i = 1; i < m; i++) pos = (pos + 1) % n;
    return pos + 1;
}
```

3. Best: direct modular formula

O(1) time, O(1) space

The first item is at K, so the last is K + M - 1 steps from the start, wrapped around the circle.

```
static int distribute(int n, int m, int k) {
    if (n <= 0 || m <= 0 || k < 1 || k > n) throw new IllegalArgumentException();
    return (int)((long)k - 1 + (long)m - 1) % n + 1;
}
```

Tips and edge cases

Do not confuse this with Josephus elimination: here items go to consecutive positions, and nobody is removed. Long arithmetic avoids overflow before modulo. If the prompt uses a different starting-index convention, adjust the formula.

GeeksforGeeks: [Open source problem / topic](#)

LeetCode matches or related practice: [Find The Winner Of The Circular Game](#) (related cyclic elimination; different task)

04. First negative integer in every window of size k

Problem: Given an integer array and k, report the first negative value in each contiguous window of length k. Return 0 for a window with no negative value, matching the supplied snippet. If $0 < k \leq n$ is not satisfied, return an empty result.

Pattern: Fixed-size sliding window; keep only candidate negative indices that have not expired.

1. Brute force: scan each window

$O(nk)$ time, $O(n-k+1)$ output space

For each window start, scan left to right until the first negative value appears.

```
static List<Integer> firstNegativeBrute(int[] a, int k) {
    List<Integer> out = new ArrayList<>();
    if (k <= 0 || k > a.length) return out;
    for (int left = 0; left + k <= a.length; left++) {
        int answer = 0;
        for (int i = left; i < left + k; i++)
            if (a[i] < 0) { answer = a[i]; break; }
        out.add(answer);
    }
    return out;
}
```

2. Better: queue negative indices

$O(n)$ time, $O(k)$ auxiliary space

Add a negative index as the right edge advances; remove indices before the current window start. The queue front is the answer.

```
static List<Integer> firstNegativeQueue(int[] a, int k) {
    List<Integer> out = new ArrayList<>();
    if (k <= 0 || k > a.length) return out;
    Deque<Integer> q = new ArrayDeque<>();
    for (int right = 0; right < a.length; right++) {
        if (a[right] < 0) q.addLast(right);
        int left = right - k + 1;
        if (left >= 0) {
            while (!q.isEmpty() && q.peekFirst() < left) q.removeFirst();
            out.add(q.isEmpty() ? 0 : a[q.peekFirst()]);
        }
    }
    return out;
}
```

3. Best for this exact task: one forward pointer

$O(n)$ time, $O(1)$ auxiliary space

The first-negative index only moves forward. This is the optimized idea in the pasted code, with input validation and explicit in-window checks.

```
static List<Integer> firstNegativePointer(int[] a, int k) {
    List<Integer> out = new ArrayList<>();
    int n = a.length;
    if (k <= 0 || k > n) return out;
    int first = 0;
    for (int right = k - 1; right < n; right++) {
        int left = right - k + 1;
        while (first < right && (first < left || a[first] >= 0)) first++;
        out.add(first <= right && a[first] < 0 ? a[first] : 0);
    }
    return out;
}
```

Tips and edge cases

The pasted fence says python, but the body is Java. Keep index queues rather than values so expired duplicates can be removed correctly. Use long for sums in related fixed-window problems.

GeeksforGeeks: [Open source problem / topic](#)

LeetCode matches or related practice: [Sliding Window Maximum](#) • [Maximum Average Subarray I](#)

05. Count distinct elements in every window of size k

Problem: For each contiguous window of length k, return the number of different values it contains. Example: [1,2,1,3,3], k=3 gives [2,3,2].

Pattern: Fixed window plus a frequency map; a value is distinct in the window exactly while its frequency is positive.

1. Brute force: compare each value with earlier values

O(nk²) time, O(1) auxiliary space

For every window position, count a value only if it has not appeared earlier in that same window.

```
static List<Integer> distinctBrute(int[] a, int k) {
    List<Integer> out = new ArrayList<>();
    if (k <= 0 || k > a.length) return out;
    for (int left = 0; left + k <= a.length; left++) {
        int count = 0;
        for (int i = left; i < left + k; i++) {
            boolean seen = false;
            for (int j = left; j < i; j++)
                if (a[j] == a[i]) { seen = true; break; }
            if (!seen) count++;
        }
        out.add(count);
    }
    return out;
}
```

2. Better: build a set for each window

O(nk) expected time, O(k) space

A HashSet removes duplicate values inside each window, but rebuilding it repeats work.

```
static List<Integer> distinctSetPerWindow(int[] a, int k) {
    List<Integer> out = new ArrayList<>();
    if (k <= 0 || k > a.length) return out;
    for (int left = 0; left + k <= a.length; left++) {
        Set<Integer> set = new HashSet<>();
        for (int i = left; i < left + k; i++) set.add(a[i]);
        out.add(set.size());
    }
    return out;
}
```

3. Best: slide a frequency map

O(n) expected time, O(k) space

Add the incoming value and remove the outgoing one. When a count falls to zero, remove the key.

```
static List<Integer> distinctSliding(int[] a, int k) {
    List<Integer> out = new ArrayList<>();
    if (k <= 0 || k > a.length) return out;
    Map<Integer, Integer> freq = new HashMap<>();
    for (int i = 0; i < k; i++) freq.merge(a[i], 1, Integer::sum);
    out.add(freq.size());
    for (int right = k; right < a.length; right++) {
        int leaving = a[right - k];
        int count = freq.get(leaving) - 1;
        if (count == 0) freq.remove(leaving); else freq.put(leaving, count);
        freq.merge(a[right], 1, Integer::sum);
        out.add(freq.size());
    }
    return out;
}
```

Tips and edge cases

The GfG page title is about distinct array values. The pasted countAnagram method below solves a different string-window task. If integer values fall in a small known range, an int[] frequency table can replace HashMap.

GeeksforGeeks: [Open source problem / topic](#)

LeetCode matches or related practice: [Maximum Sum Of Distinct Subarrays With Length K](#) • [Longest Substring Without Repeating Characters](#)

06. Count distinct anagram substrings of a fixed length

Problem: Given text and word, count the number of distinct substrings of text whose length equals word.length() and whose character multiset equals word. “Distinct” means identical substring values count once, even if they occur at multiple positions.

Pattern: Fixed-size string window; frequency equality; HashSet for unique matching substrings.

1. Brute force: sort every window

$O((n-m+1) m \log m)$ time, $O(m)$ auxiliary space

Sort the pattern once and sort each length- m window; store matching strings in a set.

```
static int countAnagramBrute(String text, String word) {
    int n = text.length(), m = word.length();
    if (m == 0 || m > n) return 0;
    char[] target = word.toCharArray(); Arrays.sort(target);
    Set<String> found = new HashSet<>();
    for (int left = 0; left + m <= n; left++) {
        char[] w = text.substring(left, left + m).toCharArray();
        Arrays.sort(w);
        if (Arrays.equals(target, w)) found.add(text.substring(left, left + m));
    }
    return found.size();
}
```

2. Better: compare 26-letter frequency arrays

$O(26n)$ time, $O(26 + \text{output})$ space

Update one outgoing and one incoming count per slide. Compare the fixed alphabet table at each window.

```
static int countAnagramFreq(String text, String word) {
    int n = text.length(), m = word.length();
    if (m == 0 || m > n) return 0;
    int[] need = new int[26], have = new int[26];
    for (int i = 0; i < m; i++) {
        need[word.charAt(i) - 'a']++;
        have[text.charAt(i) - 'a']++;
    }
    Set<String> found = new HashSet<>();
    for (int right = m - 1; right < n; right++) {
        if (Arrays.equals(need, have))
            found.add(text.substring(right - m + 1, right + 1));
        if (right + 1 < n) {
            have[text.charAt(right - m + 1) - 'a']--;
            have[text.charAt(right + 1) - 'a']++;
        }
    }
    return found.size();
}
```

3. Best: mismatch counter plus set

$O(n + r \cdot m)$ expected time, $O(26 + r \cdot m)$ space for r matching windows of length m

Track how many letters still have a nonzero frequency difference. A matching window has mismatch count 0.

```
static int countAnagramFast(String text, String word) {
    int n = text.length(), m = word.length();
    if (m == 0 || m > n) return 0;
    int[] diff = new int[26];
    for (int i = 0; i < m; i++) diff[word.charAt(i) - 'a']++;
    int mismatches = 0;
    for (int x : diff) if (x != 0) mismatches++;
    for (int i = 0; i < m; i++)
        mismatches = change(diff, text.charAt(i) - 'a', -1, mismatches);
    Set<String> found = new HashSet<>();
    for (int right = m - 1; right < n; right++) {
        if (mismatches == 0)
            found.add(text.substring(right - m + 1, right + 1));
        if (right + 1 < n) {
            mismatches = change(diff, text.charAt(right - m + 1) - 'a', 1, mismatches);
            mismatches = change(diff, text.charAt(right + 1) - 'a', -1, mismatches);
        }
    }
    return found.size();
}

static int change(int[] diff, int c, int delta, int mismatches) {
    if (diff[c] == 0) mismatches++;
    diff[c] += delta;
    if (diff[c] == 0) mismatches--;
    return mismatches;
}
```

Tips and edge cases

The pasted countAnagram code counts distinct matching substrings. Its 256-slot tables can crash for Java characters above U+00FF; this workbook uses lowercase English letters. To count every occurrence instead, remove the HashSet and increment an integer.

LeetCode matches or related practice: [Find All Anagrams In A String](#) • [Permutation In String](#)

07. Check whether a string contains a permutation of another

Problem: Given strings $s1$ and $s2$, return true if some permutation of $s1$ appears as a contiguous substring of $s2$; otherwise return false. The matching substring must have length $s1.length()$.

Pattern: Fixed-size sliding window plus character-frequency matching.

1. Brute force: sort every candidate substring

$O((n-m+1) m \log m)$ time, $O(m)$ space

Sort $s1$ once. Sort every length- m window in $s2$ and compare.

```
static boolean containsPermutationBrute(String s1, String s2) {
    int m = s1.length(), n = s2.length();
    if (m > n) return false;
    char[] target = s1.toCharArray(); Arrays.sort(target);
    for (int left = 0; left + m <= n; left++) {
        char[] w = s2.substring(left, left + m).toCharArray();
        Arrays.sort(w);
        if (Arrays.equals(target, w)) return true;
    }
    return false;
}
```

2. Better: compare frequency arrays

$O(26n)$ time, $O(26)$ space

Maintain one fixed-length window and compare its 26 counts with $s1$.

```
static boolean containsPermutationFreq(String s1, String s2) {
    int m = s1.length(), n = s2.length();
    if (m > n) return false;
    int[] need = new int[26], have = new int[26];
    for (int i = 0; i < m; i++) {
        need[s1.charAt(i) - 'a']++;
        have[s2.charAt(i) - 'a']++;
    }
    for (int right = m - 1; right < n; right++) {
        if (Arrays.equals(need, have)) return true;
        if (right + 1 < n) {
            have[s2.charAt(right - m + 1) - 'a']--;
            have[s2.charAt(right + 1) - 'a']++;
        }
    }
    return false;
}
```


3. Best: mismatch counter

$O(n + 26)$ time, $O(26)$ space

Update only two counts per slide and test whether every frequency matches.

```
static boolean containsPermutationFast(String s1, String s2) {
    int m = s1.length(), n = s2.length();
    if (m > n) return false;
    int[] d = new int[26];
    for (int i = 0; i < m; i++) {
        d[s1.charAt(i) - 'a']++;
        d[s2.charAt(i) - 'a']--;
    }
    int bad = 0;
    for (int x : d) if (x != 0) bad++;
    if (bad == 0) return true;
    for (int right = m; right < n; right++) {
        bad = change(d, s2.charAt(right - m) - 'a', 1, bad);
        bad = change(d, s2.charAt(right) - 'a', -1, bad);
        if (bad == 0) return true;
    }
    return false;
}

static int change(int[] d, int c, int delta, int bad) {
    if (d[c] == 0) bad++;
    d[c] += delta;
    if (d[c] == 0) bad--;
    return bad;
}
```

Tips and edge cases

This is a boolean decision problem; it is different from counting distinct anagram windows. If the alphabet is not only lowercase a-z, use a HashMap or a larger Unicode-aware structure.

GeeksforGeeks: [Open source problem / topic](#)

LeetCode matches or related practice: [Find All Anagrams In A String](#)

08. Find a subarray with a given sum

Problem: Return the first zero-based inclusive [left, right] range whose elements sum to target, or an empty array if none exists. The general version may contain negative numbers. A separate $O(1)$ -space sliding-window method is valid only when every element is nonnegative.

Pattern: Prefix sums and hashing for arbitrary integers; variable-size sliding window when values are nonnegative.

1. Brute force: try every start and end

$O(n^2)$ time, $O(1)$ auxiliary space

Accumulate each subarray sum until a match is found.

```
static int[] findSumBrute(int[] a, long target) {
    for (int left = 0; left < a.length; left++) {
        long sum = 0;
        for (int right = left; right < a.length; right++) {
            sum += a[right];
            if (sum == target) return new int[]{left, right};
        }
    }
    return new int[0];
}
```

2. Better: prefix sums with an earliest-index map

$O(n)$ expected time, $O(n)$ space

At index r , a previous prefix p gives target sum when $\text{currentPrefix} - p = \text{target}$.

```
static int[] findSumPrefix(int[] a, long target) {
    Map<Long, Integer> earliest = new HashMap<>();
    earliest.put(0L, -1);
    long prefix = 0;
    for (int right = 0; right < a.length; right++) {
        prefix += a[right];
        Integer before = earliest.get(prefix - target);
        if (before != null) return new int[]{before + 1, right};
        earliest.putIfAbsent(prefix, right);
    }
    return new int[0];
}
```

3. Best for nonnegative arrays: sliding window

$O(n)$ time, $O(1)$ auxiliary space

Expand right; while the sum is too large, advance left. Do not use this when negative values are possible.

```
static int[] findSumNonnegative(int[] a, long target) {
    int left = 0;
    long sum = 0;
    for (int right = 0; right < a.length; right++) {
        sum += a[right];
        while (left <= right && sum > target) sum -= a[left++];
        if (sum == target && left <= right) return new int[]{left, right};
    }
    return new int[0];
}
```

Tips and edge cases

The GfG problem is commonly stated with nonnegative values; prefix sums are the safe generalization. Use long for prefix sums. Check the exact required output convention (1-based vs 0-based, and whether empty subarrays are allowed).

GeeksforGeeks: [Open source problem / topic](#)

LeetCode matches or related practice: [Subarray Sum Equals K](#) • [Minimum Size Subarray Sum](#)

09. Pattern searching: find every occurrence of a pattern

Problem: Given text and pattern strings, return the starting index of every occurrence of pattern in text. An empty pattern is treated as matching at index 0 through `text.length()`.

Pattern: String matching; reuse work from previous comparisons instead of restarting at every text index.

1. Brute force: compare at every alignment

$O((n-m+1)m)$ time, $O(1)$ extra space

Try each possible start and compare characters until a mismatch.

```
static List<Integer> searchBrute(String text, String pat) {
    List<Integer> out = new ArrayList<>();
    int n = text.length(), m = pat.length();
    if (m == 0) { out.add(0); return out; }
    for (int s = 0; s + m <= n; s++) {
        int j = 0;
        while (j < m && text.charAt(s+j) == pat.charAt(j)) j++;
        if (j == m) out.add(s);
    }
    return out;
}
```

2. Better: Rabin-Karp rolling hash

$O(n+m)$ average, $O(nm)$ worst-case time; $O(1)$ extra space

Hash each length- m window in $O(1)$ updates. Verify characters when hashes match to handle collisions.

```
static List<Integer> searchRabinKarp(String t, String p) {
    List<Integer> out = new ArrayList<>();
    int n = t.length(), m = p.length();
    if (m == 0) { out.add(0); return out; }
    if (m > n) return out;
    long mod = 1_000_000_007L, base = 257, power = 1;
    long ph = 0, wh = 0;
    for (int i = 0; i < m; i++) {
        ph = (ph * base + p.charAt(i)) % mod;
        wh = (wh * base + t.charAt(i)) % mod;
        if (i < m - 1) power = power * base % mod;
    }
    for (int start = 0; start + m <= n; start++) {
        if (ph == wh && t.regionMatches(start, p, 0, m)) out.add(start);
        if (start + m < n) {
            wh = (wh - t.charAt(start) * power % mod + mod) % mod;
            wh = (wh * base + t.charAt(start + m)) % mod;
        }
    }
    return out;
}
```

3. Best worst-case guarantee: KMP

$O(n+m)$ time, $O(m)$ space

The LPS table tells how far the pattern can shift after a mismatch without rescanning matched text.

```
static List<Integer> searchKmp(String t, String p) {
    List<Integer> out = new ArrayList<>();
    int n = t.length(), m = p.length();
    if (m == 0) { out.add(0); return out; }
    int[] lps = new int[m];
    for (int i = 1, len = 0; i < m; i++) {
        if (p.charAt(i) == p.charAt(len)) lps[i] = ++len;
        else if (len > 0) len = lps[len - 1];
        else lps[i] = 0;
    }
    for (int i = 0, j = 0; i < n; i++) {
        if (t.charAt(i) == p.charAt(j)) { i++; j++; }
        if (j == m) { out.add(i - j); j = lps[j - 1]; }
        else if (i < n && t.charAt(i) != p.charAt(j)) {
            if (j > 0) j = lps[j - 1]; else i++;
        }
    }
    return out;
}
```

Tips and edge cases

Rabin-Karp hashes are not proof of equality: always verify a hash match. KMP is the safe choice for predictable linear worst-case time. This user link is a topic guide, so the workbook uses the standard all-occurrences search task.

GeeksforGeeks: [Open source problem / topic](#)

LeetCode matches or related practice: [Find The Index Of The First Occurrence In A String](#) • [Repeated Dna Sequences](#)

10. Two pointers: pair with a target sum in a sorted array

Problem: Given a nondecreasing array and target, return any pair of distinct indices whose values sum to target, or an empty array. This is a representative problem for the supplied two-pointers technique guide.

Pattern: Two pointers; sorting order tells which pointer can move without missing a solution.

1. Brute force: check every pair

$O(n^2)$ time, $O(1)$ extra space

Try all $i < j$.

```
static int[] pairBrute(int[] a, int target) {
    for (int i = 0; i < a.length; i++)
        for (int j = i + 1; j < a.length; j++)
            if ((long)a[i] + a[j] == target) return new int[]{i, j};
    return new int[0];
}
```

2. Better: complement HashMap

$O(n)$ expected time, $O(n)$ space

Remember the earliest index for each value; look for target - current.

```
static int[] pairHash(int[] a, int target) {
    Map<Long, Integer> seen = new HashMap<>();
    for (int i = 0; i < a.length; i++) {
        long need = (long)target - a[i];
        if (seen.containsKey(need)) return new int[]{seen.get(need), i};
        seen.putIfAbsent((long)a[i], i);
    }
    return new int[0];
}
```

3. Best when the array is sorted: left/right pointers

$O(n)$ time, $O(1)$ extra space

If sum is too small, increase the left value. If too large, decrease the right value.

```
static int[] pairTwoPointers(int[] a, int target) {
    int left = 0, right = a.length - 1;
    while (left < right) {
        long sum = (long)a[left] + a[right];
        if (sum == target) return new int[]{left, right};
        if (sum < target) left++;
        else right--;
    }
    return new int[0];
}
```

Tips and edge cases

The two-pointer method needs sorted input. If the original array is unsorted and indices matter, use a HashMap or sort value-index pairs. Use long for the sum if values may be large.

GeeksforGeeks: [Open source problem / topic](#)

LeetCode matches or related practice: [Two Sum II Input Array Is Sorted](#) • [3Sum](#)

11. Rotate an array right by d positions

Problem: Move every array value d places to the right, wrapping values from the end to the front. Example: [1,2,3,4,5], d=2 becomes [4,5,1,2,3].

Pattern: Array rotation; normalize d modulo n; reversal turns a shift into three in-place reversals.

1. Brute force: rotate one place repeatedly

O(nd) time, O(1) extra space

Repeat a one-step right shift d times.

```
static void rotateBrute(int[] a, int d) {
    int n = a.length;
    if (n == 0) return;
    d = Math.floorMod(d, n);
    for (int step = 0; step < d; step++) {
        int last = a[n - 1];
        for (int i = n - 1; i > 0; i--) a[i] = a[i - 1];
        a[0] = last;
    }
}
```

2. Better: write into a second array

O(n) time, O(n) extra space

Place each original index i at (i+d) mod n, then copy back.

```
static void rotateBuffer(int[] a, int d) {
    int n = a.length;
    if (n == 0) return;
    d = Math.floorMod(d, n);
    int[] b = new int[n];
    for (int i = 0; i < n; i++) b[(i + d) % n] = a[i];
    System.arraycopy(b, 0, a, 0, n);
}
```

3. Best: reverse three sections

O(n) time, O(1) extra space

Reverse all, reverse the first d values, then reverse the remaining suffix.

```
static void rotateReverse(int[] a, int d) {
    int n = a.length;
    if (n == 0) return;
    d = Math.floorMod(d, n);
    reverse(a, 0, n - 1);
    reverse(a, 0, d - 1);
    reverse(a, d, n - 1);
}

static void reverse(int[] a, int l, int r) {
    while (l < r) { int t = a[l]; a[l++] = a[r]; a[r--] = t; }
}
```

Tips and edge cases

For a left rotation by d, use right distance n - d after normalization. Test d=0, d=n, d>n, and an empty array. This rotates the array; shell rotation of matrix rings is a separate problem below.

GeeksforGeeks: [Open source problem / topic](#)

LeetCode matches or related practice: [Rotate Array](#)

12. Rotate every matrix ring clockwise by one position

Problem: Given an $m \times n$ matrix, shift each outer and inner ring by one cell clockwise. For example, the top-left value moves one cell right; the center row or center column of an odd-sized matrix stays in place.

Pattern: Layer traversal; represent each ring in clockwise order and shift its values cyclically.

1. Brute force: copy the whole matrix

$O(mn)$ time, $O(mn)$ extra space

Copy the matrix, then for each ring copy every value from the previous clockwise cell in the original matrix.

```
static void rotateRingsCopy(int[][] a) {
    if (a.length == 0 || a[0].length == 0) return;
    int m = a.length, n = a[0].length;
    int[][] old = new int[m][n];
    for (int r = 0; r < m; r++) old[r] = a[r].clone();
    for (int top = 0, left = 0, bottom = m - 1, right = n - 1;
         top < bottom && left < right;
         top++, left++, bottom--, right--) {
        for (int c = left + 1; c <= right; c++) a[top][c] = old[top][c - 1];
        for (int r = top + 1; r <= bottom; r++) a[r][right] = old[r - 1][right];
        for (int c = right - 1; c >= left; c--) a[bottom][c] = old[bottom][c + 1];
        for (int r = bottom - 1; r > top; r--) a[r][left] = old[r + 1][left];
        a[top][left] = old[top + 1][left];
    }
}
```

2. Better: buffer one ring at a time

$O(mn)$ time, $O(\min(m,n))$ peak extra space

Store one perimeter in clockwise order, then write each value to the next position. A single ring buffer is reused.

```
static void rotateRingsBuffered(int[][] a) {
    if (a.length == 0 || a[0].length == 0) return;
    int m = a.length, n = a[0].length;
    for (int top = 0, left = 0, bottom = m - 1, right = n - 1;
         top < bottom && left < right;
         top++, left++, bottom--, right--) {
        int len = 2 * (bottom - top + right - left);
        int[] ring = new int[len]; int p = 0;
        for (int c = left; c <= right; c++) ring[p++] = a[top][c];
        for (int r = top + 1; r <= bottom; r++) ring[p++] = a[r][right];
        for (int c = right - 1; c >= left; c--) ring[p++] = a[bottom][c];
        for (int r = bottom - 1; r > top; r--) ring[p++] = a[r][left];
        p = 0;
        for (int c = left + 1; c <= right; c++) a[top][c] = ring[p++];
        for (int r = top + 1; r <= bottom; r++) a[r][right] = ring[p++];
        for (int c = right - 1; c >= left; c--) a[bottom][c] = ring[p++];
        for (int r = bottom - 1; r > top; r--) a[r][left] = ring[p++];
        a[top][left] = ring[len - 1];
    }
}
```

3. Best: rotate each ring in place with one carry value

$O(mn)$ time, $O(1)$ extra space

Start with the last cell in clockwise traversal as the carry, then walk the ring clockwise, swapping the carry into each next cell.

```
static void rotateRingsInPlace(int[][] a) {
    if (a.length == 0 || a[0].length == 0) return;
    int m = a.length, n = a[0].length;
    for (int top = 0, left = 0, bottom = m - 1, right = n - 1;
        top < bottom && left < right;
        top++, left++, bottom--, right--) {
        int carry = a[top + 1][left];
        int old = a[top][left]; a[top][left] = carry; carry = old;
        for (int c = left + 1; c <= right; c++) {
            old = a[top][c]; a[top][c] = carry; carry = old;
        }
        for (int r = top + 1; r <= bottom; r++) {
            old = a[r][right]; a[r][right] = carry; carry = old;
        }
        for (int c = right - 1; c >= left; c--) {
            old = a[bottom][c]; a[bottom][c] = carry; carry = old;
        }
        for (int r = bottom - 1; r > top; r--) {
            old = a[r][left]; a[r][left] = carry; carry = old;
        }
    }
}
```

Tips and edge cases

Trace a 3x3 or 4x4 matrix on paper. Traverse each ring clockwise exactly once; the carry starts at the old last cell so the last value wraps to the top-left. Handle empty/ragged matrices per the required interface.

GeeksforGeeks: [Open source problem / topic](#)

LeetCode matches or related practice: [Spiral Matrix](#) • [Rotate Image](#)

13. Find the Nth Fibonacci number

Problem: Return $F(n)$, where $F(0)=0$, $F(1)=1$, and $F(n)=F(n-1)+F(n-2)$ for $n \geq 2$.

Pattern: Dynamic programming on a one-dimensional recurrence; retain only the two previous states.

1. Brute force: direct recursion

$O(2^n)$ time, $O(n)$ call stack

The direct recurrence recomputes the same smaller Fibonacci values many times.

```
static long fibBrute(int n) {
    if (n < 0) throw new IllegalArgumentException();
    if (n < 2) return n;
    return fibBrute(n - 1) + fibBrute(n - 2);
}
```

2. Better: memoized recursion

$O(n)$ time, $O(n)$ space

Cache every result so each state is solved once.

```
static long fibMemo(int n) {
    if (n < 0) throw new IllegalArgumentException();
    long[] memo = new long[n + 1];
    Arrays.fill(memo, -1);
    return fibMemo(n, memo);
}
static long fibMemo(int n, long[] memo) {
    if (n < 2) return n;
    if (memo[n] != -1) return memo[n];
    return memo[n] = fibMemo(n - 1, memo) + fibMemo(n - 2, memo);
}
```

3. Best: iterative two-variable DP

$O(n)$ time, $O(1)$ extra space

Walk upward and keep only $F(i-1)$ and $F(i)$.

```
static long fib(int n) {
    if (n < 0) throw new IllegalArgumentException();
    long prev = 0, curr = 1;
    for (int i = 0; i < n; i++) {
        long next = prev + curr;
        prev = curr; curr = next;
    }
    return prev;
}
```

Tips and edge cases

Java long overflows after $F(92)$. If the prompt asks for larger n , use `BigInteger` or the required modulus. Keep the base cases explicit before looping.

GeeksforGeeks: [Open source problem / topic](#)

LeetCode matches or related practice: [Fibonacci Number](#) • [N Th Tribonacci Number](#)

14. 0/1 Knapsack

Problem: Given weights and values for n items and a capacity W , choose each item at most once to maximize total value without exceeding W .

Pattern: Dynamic programming over item index and remaining capacity; for each item choose skip or take.

1. Brute force: enumerate choices

$O(2^n)$ time, $O(n)$ call stack

At each item, branch into taking it or skipping it.

```
static int knapsackBrute(int i, int cap, int[] wt, int[] val) {
    if (i == wt.length || cap == 0) return 0;
    int skip = knapsackBrute(i + 1, cap, wt, val);
    if (wt[i] > cap) return skip;
    return Math.max(skip, val[i] + knapsackBrute(i + 1, cap - wt[i], wt, val));
}
```

2. Better: memoize item and capacity states

$O(nW)$ time, $O(nW)$ space

Cache the best value for each pair (item index, remaining capacity).

```
static int knapsackMemo(int[] wt, int[] val, int W) {
    int[][] memo = new int[wt.length][W + 1];
    for (int[] row : memo) Arrays.fill(row, -1);
    return solveKnapsack(0, W, wt, val, memo);
}
static int solveKnapsack(int i, int cap, int[] wt, int[] val, int[][] memo) {
    if (i == wt.length || cap == 0) return 0;
    if (memo[i][cap] != -1) return memo[i][cap];
    int best = solveKnapsack(i + 1, cap, wt, val, memo);
    if (wt[i] <= cap) best = Math.max(best, val[i] + solveKnapsack(i + 1, cap - wt[i], wt, val, memo));
    return memo[i][cap] = best;
}
```

3. Best: 1D bottom-up DP

$O(nW)$ time, $O(W)$ space

Process capacities downward so the current item cannot be reused within the same iteration.

```
static int knapsack(int[] wt, int[] val, int W) {
    int[] dp = new int[W + 1];
    for (int i = 0; i < wt.length; i++)
        for (int cap = W; cap >= wt[i]; cap--)
            dp[cap] = Math.max(dp[cap], val[i] + dp[cap - wt[i]]);
    return dp[W];
}
```

Tips and edge cases

The descending capacity loop is the key 0/1 detail; ascending order changes the problem into unbounded knapsack. For very large W , use a different method if constraints allow. Values and weights are assumed nonnegative.

GeeksforGeeks: [Open source problem / topic](#)

LeetCode matches or related practice: [Partition Equal Subset Sum](#) • [Target Sum](#) • [Coin Change II](#)

15. Dynamic programming pattern guide: count ways to climb stairs

Problem: The supplied DP link is a topic guide, not one specific problem. Use this small representative task: from step 0, reach step n by taking either 1 or 2 steps. Return the number of distinct ways.

Pattern: DP checklist: define the state, write a transition, set base cases, choose memoization or tabulation, then optimize stored states.

1. Brute force: branch over each next step

$O(2^n)$ time, $O(n)$ call stack

The number of ways from n is the sum of ways from $n-1$ and $n-2$.

```
static long climbBrute(int n) {
    if (n < 0) return 0;
    if (n == 0 || n == 1) return 1;
    return climbBrute(n - 1) + climbBrute(n - 2);
}
```

2. Better: memoized recurrence

$O(n)$ time, $O(n)$ space

Store each computed state to avoid solving the same subproblem again.

```
static long climbMemo(int n) {
    long[] memo = new long[n + 1];
    Arrays.fill(memo, -1);
    return climbMemo(n, memo);
}
static long climbMemo(int n, long[] memo) {
    if (n < 0) return 0;
    if (n <= 1) return 1;
    if (memo[n] != -1) return memo[n];
    return memo[n] = climbMemo(n - 1, memo) + climbMemo(n - 2, memo);
}
```

3. Best for this recurrence: bottom-up with two states

$O(n)$ time, $O(1)$ extra space

Fill from smaller states and retain the previous two values only.

```
static long climbStairs(int n) {
    if (n < 0) return 0;
    long oneBack = 1, twoBack = 1;
    for (int step = 2; step <= n; step++) {
        long ways = oneBack + twoBack;
        twoBack = oneBack; oneBack = ways;
    }
    return oneBack;
}
```

Tips and edge cases

Before coding a DP problem, write: state meaning, recurrence, base cases, evaluation order, and answer location. Fibonacci and stairs are one-dimensional; knapsack adds a capacity dimension. Use BigInteger/modulo when counts can grow.

GeeksforGeeks: [Open source problem / topic](#)

LeetCode matches or related practice: [Climbing Stairs](#) • [House Robber](#)

16. Minimum Fibonacci numbers whose sum is K

Problem: Given positive integer K, return the fewest Fibonacci numbers (repetitions allowed) whose sum is exactly K. The standard greedy solution uses the Fibonacci sequence 1, 1, 2, 3, 5, ... and repeatedly takes the largest value not exceeding the remainder.

Pattern: Greedy choice with Fibonacci denominations; subtract the largest feasible term repeatedly.

1. Brute force: recurse over available terms

Exponential time in K, $O(\log K)$ recursion depth

Try using or skipping each Fibonacci denomination and retain the smallest number of terms. This is practical only for small K.

```
static int fibTermsBrute(int k, int[] f, int i) {
    if (k == 0) return 0;
    if (i < 0 || k < 0) return 1_000_000;
    int use = fibTermsBrute(k - f[i], f, i);
    if (use < 1_000_000) use++;
    int skip = fibTermsBrute(k, f, i - 1);
    return Math.min(use, skip);
}
```

2. Better: coin-change DP over all sums

$O(K \log K)$ time, $O(K)$ space

Treat Fibonacci numbers as reusable denominations and compute the minimum terms for every sum up to K.

```
static int fibTermsDp(int k) {
    List<Integer> f = fibonacciUpTo(k);
    int[] dp = new int[k + 1];
    Arrays.fill(dp, k + 1); dp[0] = 0;
    for (int coin : f)
        for (int sum = coin; sum <= k; sum++)
            dp[sum] = Math.min(dp[sum], dp[sum - coin] + 1);
    return dp[k];
}
```

3. Best: largest-term greedy

$O(\log K)$ time, $O(\log K)$ Fibonacci list

The Fibonacci denomination system has the greedy minimum-term property. Generate terms up to K and subtract the largest one repeatedly.

```
static List<Integer> fibonacciUpTo(int k) {
    List<Integer> f = new ArrayList<>(Arrays.asList(1, 2));
    while (f.get(f.size() - 1) <= k - f.get(f.size() - 2))
        f.add(f.get(f.size() - 1) + f.get(f.size() - 2));
    return f;
}
static int minFibonacciTerms(int k) {
    if (k < 0) throw new IllegalArgumentException();
    List<Integer> f = fibonacciUpTo(k);
    int count = 0;
    for (int i = f.size() - 1; i >= 0; i--) {
        count += k / f.get(i);
        k %= f.get(i);
    }
    return count;
}
```

Tips and edge cases

Fibonacci terms may be repeated in the general coin-change statement; the greedy algorithm still gives the minimum for Fibonacci denominations. Guard integer overflow when constraints exceed int. K=0 returns zero.

GeeksforGeeks: [Open source problem / topic](#)

LeetCode matches or related practice: [Find The Minimum Number Of Fibonacci Numbers Whose Sum Is K](#)

17. Minimum number of coins for a value

Problem: Given positive coin denominations and amount A, return the minimum number of coins needed to form A, or -1 if it is impossible. Greedy is optimal for the standard Indian denomination set, but not for every arbitrary set.

Pattern: Coin change; compare the denomination-system-specific greedy rule with the general dynamic-programming optimum.

1. Brute force: try every usable coin

Exponential time, $O(A)$ call stack

For each amount, try each coin and keep the smallest recursive answer.

```
static int minCoinsBrute(int amount, int[] coins) {
    if (amount == 0) return 0;
    int best = amount + 1;
    for (int coin : coins) if (coin <= amount) {
        int sub = minCoinsBrute(amount - coin, coins);
        if (sub >= 0) best = Math.min(best, sub + 1);
    }
    return best == amount + 1 ? -1 : best;
}
```

2. Better: bottom-up coin-change DP

$O(A * \text{numberOfCoins})$ time, $O(A)$ space

For each subtotal, store the fewest coins needed so far. This works for arbitrary positive denominations.

```
static int minCoinsDp(int amount, int[] coins) {
    int[] dp = new int[amount + 1];
    Arrays.fill(dp, amount + 1); dp[0] = 0;
    for (int sum = 1; sum <= amount; sum++)
        for (int coin : coins) if (coin <= sum)
            dp[sum] = Math.min(dp[sum], dp[sum - coin] + 1);
    return dp[amount] > amount ? -1 : dp[amount];
}
```

3. Best for the canonical Indian coin set: greedy

$O(C \log C)$ time if sorting, $O(1)$ extra space

Sort denominations descending, take as many as possible of each. Use only when the coin system is known to be canonical.

```
static List<Integer> greedyCoins(int amount, int[] coins) {
    Integer[] c = Arrays.stream(coins).boxed().toArray(Integer[]::new);
    Arrays.sort(c, Collections.reverseOrder());
    List<Integer> used = new ArrayList<>();
    for (int coin : c) while (amount >= coin) {
        amount -= coin; used.add(coin);
    }
    if (amount != 0) return Collections.emptyList();
    return used;
}
```

Tips and edge cases

Counterexample for arbitrary coins: denominations [1,3,4], amount 6. Greedy picks 4+1+1 (3 coins), while 3+3 uses

2. Verify the coin system or use DP when optimality is required.

GeeksforGeeks: [Open source problem / topic](#)

LeetCode matches or related practice: [Coin Change](#) • [Coin Change II](#)

18. Minimum platforms for a railway station

Problem: Given train arrival and departure times for one day, find the minimum number of platforms so no train waits. This workbook uses inclusive occupancy: a train arriving at the same recorded time another departs still needs a separate platform, matching the common GFG convention.

Pattern: Interval overlap; sort arrivals and departures, then sweep the next event.

1. Brute force: count trains present at each arrival

$O(n^2)$ time, $O(1)$ extra space

At each arrival time, count every interval that overlaps it; the largest overlap is the platform requirement.

```
static int platformsBrute(int[] arr, int[] dep) {
    int best = 0;
    for (int time : arr) {
        int count = 0;
        for (int i = 0; i < arr.length; i++)
            if (arr[i] <= time && time <= dep[i]) count++;
        best = Math.max(best, count);
    }
    return best;
}
```

2. Better: sort all arrival/departure events

$O(n^2)$ time in this simple sort, $O(n)$ space

Sort event times; at equal time, process arrivals before departures for inclusive occupancy.

```
static int platformsEvents(int[] arr, int[] dep) {
    int n = arr.length;
    int[] time = new int[2 * n]; int[] type = new int[2 * n];
    for (int i = 0; i < n; i++) {
        time[2 * i] = arr[i]; type[2 * i] = 1;
        time[2 * i + 1] = dep[i]; type[2 * i + 1] = -1;
    }
    for (int i = 0; i < 2 * n; i++) for (int j = i + 1; j < 2 * n; j++)
        if (time[j] < time[i] || (time[j] == time[i] && type[j] > type[i])) {
            int t = time[i]; time[i] = time[j]; time[j] = t;
            int x = type[i]; type[i] = type[j]; type[j] = x;
        }
    int active = 0, best = 0;
    for (int t : type) { active += t; best = Math.max(best, active); }
    return best;
}
```

3. Best: two sorted arrays and two pointers

$O(n \log n)$ time, $O(1)$ extra space after sorting copies

If the next arrival is no later than the next departure, allocate another platform; otherwise release one.

```
static int minPlatforms(int[] arrivals, int[] departures) {
    if (arrivals.length == 0) return 0;
    int[] arr = arrivals.clone(), dep = departures.clone();
    Arrays.sort(arr); Arrays.sort(dep);
    int i = 0, j = 0, active = 0, best = 0;
    while (i < arr.length) {
        if (arr[i] <= dep[j]) {
            active++; best = Math.max(best, active); i++;
        } else {
            active--; j++;
        }
    }
    return best;
}
```

Tips and edge cases

Validate the simultaneous arrival/departure rule with your instructor. If the station reuses a platform at the exact same timestamp, change `<=` to `<` or process departures first. Arrival/departure arrays must describe valid same-day intervals.

GeeksforGeeks: [Open source problem / topic](#)

LeetCode matches or related practice: [Meeting Rooms Ii](#) • [Meeting Rooms Iii](#)

19. Build the smallest number by removing k digits

Problem: Given a nonnegative decimal string num and integer k, remove exactly k digits while preserving the order of the remaining digits. Return the smallest possible number without leading zeroes; return "0" if no digits remain.

Pattern: Monotonic increasing stack; when a smaller incoming digit can replace a larger previous digit, remove the larger digit.

1. Brute force: try every deletion choice

$O(C(n,k) * n)$ time, exponential in n

Enumerate each choice of k removed positions, normalize the remaining string, and keep the smallest.

```
static String removeKBrute(String num, int k) {
    return removeKBrute(num, k, 0, new StringBuilder(), null);
}
static String removeKBrute(String s, int k, int i, StringBuilder kept, String best) {
    int remaining = s.length() - i;
    if (remaining < k) return best;
    if (i == s.length()) {
        if (k != 0) return best;
        String cand = normalizeNumber(kept.toString());
        return best == null || cand.compareTo(best) < 0 ? cand : best;
    }
    String ans = removeKBrute(s, k - 1, i + 1, kept, best);
    kept.append(s.charAt(i));
    ans = removeKBrute(s, k, i + 1, kept, ans);
    kept.setLength(kept.length() - 1);
    return ans;
}
static String normalizeNumber(String s) {
    int i = 0; while (i < s.length() && s.charAt(i) == '0') i++;
    return i == s.length() ? "0" : s.substring(i);
}
```

2. Better: repeatedly delete the first descending adjacent pair

$O(kn)$ time, $O(n)$ space

For each deletion, remove the left digit in the first pair where left > right; otherwise remove the last digit.

```
static String removeKRepeated(String num, int k) {
    StringBuilder s = new StringBuilder(num);
    while (k-- > 0 && s.length() > 0) {
        int i = 0;
        while (i + 1 < s.length() && s.charAt(i) <= s.charAt(i + 1)) i++;
        if (i + 1 == s.length()) i--;
        s.deleteCharAt(i);
    }
    return normalizeNumber(s.toString());
}
```

3. Best: monotonic stack

$O(n)$ time, $O(n)$ space

Pop larger previous digits while deletions remain, then trim any remaining deletions from the end.

```
static String removeKDigits(String num, int k) {
    StringBuilder st = new StringBuilder();
    for (char d : num.toCharArray()) {
        while (k > 0 && st.length() > 0 && st.charAt(st.length() - 1) > d) {
            st.setLength(st.length() - 1); k--;
        }
        st.append(d);
    }
    while (k-- > 0 && st.length() > 0) st.setLength(st.length() - 1);
    return normalizeNumber(st.toString());
}
```


Tips and edge cases

The stack must be monotonic nondecreasing. Equal digits need not be removed. Remove leading zeroes only after choosing digits; return "0" for empty or all-zero results.

GeeksforGeeks: [Open source problem / topic](#)

LeetCode matches or related practice: [Remove K Digits](#)

20. Lexicographically largest subsequence with each included character appearing at least k times

Problem: Given a lowercase string s and k , choose a subsequence T such that every distinct character appearing in T occurs at least k times in T . Return the lexicographically largest valid T . Example: $s = \text{"banana"}$, $k = 2$ gives "nn" .

Pattern: Greedy over character priority; take a character only when its remaining suffix contains at least k occurrences, then continue after its last occurrence.

1. Brute force: enumerate every subsequence

$O(2^n * n)$ time, $O(n)$ recursion space

Build each subsequence, count its characters, reject any nonempty character count below k , and compare valid results.

```
static String largestSubseqBrute(String s, int k) {
    return enumerate(s, k, 0, new StringBuilder(), "");
}
static String enumerate(String s, int k, int i, StringBuilder cur, String best) {
    if (i == s.length()) {
        int[] count = new int[26];
        for (int j = 0; j < cur.length(); j++) count[cur.charAt(j) - 'a']++;
        for (int c : count) if (c > 0 && c < k) return best;
        String candidate = cur.toString();
        return candidate.compareTo(best) > 0 ? candidate : best;
    }
    best = enumerate(s, k, i + 1, cur, best);
    cur.append(s.charAt(i));
    best = enumerate(s, k, i + 1, cur, best);
    cur.setLength(cur.length() - 1);
    return best;
}
```

2. Better: prune impossible branches during enumeration

Worst case $O(2^n * \text{alphabet})$, often fewer branches

Precompute suffix counts; abandon a partial choice as soon as an already-selected character can no longer reach k .

```
static String largestSubseqPruned(String s, int k) {
    int[][] suffix = new int[s.length() + 1][26];
    for (int i = s.length() - 1; i >= 0; i--) {
        suffix[i] = suffix[i + 1].clone();
        suffix[i][s.charAt(i) - 'a']++;
    }
    return searchSubseq(s, k, 0, new int[26], new StringBuilder(), "", suffix);
}
static String searchSubseq(String s, int k, int i, int[] count, StringBuilder cur,
    String best, int[][] suffix) {
    for (int c = 0; c < 26; c++)
        if (count[c] > 0 && count[c] + suffix[i][c] < k) return best;
    if (i == s.length()) {
        for (int c : count) if (c > 0 && c < k) return best;
        return cur.toString().compareTo(best) > 0 ? cur.toString() : best;
    }
    best = searchSubseq(s, k, i + 1, count, cur, best, suffix);
    int c = s.charAt(i) - 'a'; count[c]++; cur.append(s.charAt(i));
    best = searchSubseq(s, k, i + 1, count, cur, best, suffix);
    cur.setLength(cur.length() - 1); count[c]--;
    return best;
}
```

3. Best for lowercase letters: greedy by descending character

$O(26n)$ time, $O(n)$ output space

For each letter from z down to a, count it in the remaining suffix. If at least k remain, append those occurrences and move the suffix start after the last one.

```
static String largestSubsequenceAtLeastK(String s, int k) {
    if (k <= 0) throw new IllegalArgumentException();
    StringBuilder result = new StringBuilder();
    int start = 0;
    for (char ch = 'z'; ch >= 'a'; ch--) {
        int count = 0, last = -1;
        for (int i = start; i < s.length(); i++) if (s.charAt(i) == ch) {
            count++; last = i;
        }
        if (count >= k) {
            for (int i = 0; i < count; i++) result.append(ch);
            start = last + 1;
        }
    }
    return result.toString();
}
```

Tips and edge cases

The screenshot link says “at least k,” not “at most k.” This solution is for lowercase English letters, as in the source. Check the exact course judge constraints before using a larger character set; lexicographic ordering depends on that alphabet.

GeeksforGeeks: [Open source problem / topic](#)

LeetCode matches or related practice: [Remove Duplicate Letters](#) (related monotonic-stack idea; different requirement) • [Smallest Subsequence Of Distinct Characters](#) (related; each character once)

21. Water Connection Problem

Problem: Each house has at most one outgoing pipe and at most one incoming pipe. A network begins at a tank house and ends at a tap house. For every such network, report the tank house, tap house, and minimum pipe diameter along the path.

Pattern: Functional graph traversal; indegree-zero nodes are tanks, outdegree-zero nodes are taps, and the path bottleneck is the minimum edge weight.

1. Brute force: test paths from every house

$O(n \cdot p)$ time, $O(n \cdot p)$ space

Start at each house, follow its outgoing edge, and record the terminal house and minimum diameter. This repeats work if starts are not filtered.

```
static List<int[]> waterBrute(int n, int[][] pipes) {
    int[] to = new int[n + 1], dia = new int[n + 1], indeg = new int[n + 1];
    for (int[] p : pipes) { to[p[0]] = p[1]; dia[p[0]] = p[2]; indeg[p[1]]++; }
    List<int[]> ans = new ArrayList<>();
    for (int start = 1; start <= n; start++) if (indeg[start] == 0 && to[start] != 0) {
        int u = start, minD = Integer.MAX_VALUE, steps = 0;
        while (to[u] != 0 && steps++ <= n) { minD = Math.min(minD, dia[u]); u = to[u]; }
        if (steps <= n) ans.add(new int[]{start, u, minD});
    }
    return ans;
}
```

2. Better: DFS with visited nodes

$O(n \cdot p)$ time, $O(n \cdot p)$ space

Build adjacency and indegrees, begin at each tank, and follow its single outgoing path while tracking the bottleneck.

```
static List<int[]> waterDfs(int n, int[][] pipes) {
    int[] to = new int[n + 1], dia = new int[n + 1], indeg = new int[n + 1];
    for (int[] p : pipes) { to[p[0]] = p[1]; dia[p[0]] = p[2]; indeg[p[1]]++; }
    boolean[] seen = new boolean[n + 1];
    List<int[]> ans = new ArrayList<>();
    for (int s = 1; s <= n; s++) if (indeg[s] == 0 && to[s] != 0 && !seen[s]) {
        int u = s, minD = Integer.MAX_VALUE;
        while (u != 0 && !seen[u]) {
            seen[u] = true;
            if (to[u] == 0) break;
            minD = Math.min(minD, dia[u]); u = to[u];
        }
        if (u != 0 && to[u] == 0) ans.add(new int[]{s, u, minD});
    }
    return ans;
}
```

3. Best for the stated constraints: follow tank-to-tap chains

$O(n \cdot p)$ time, $O(n)$ space

Use arrays for next house, edge diameter, and indegree. A source has indegree zero and an outgoing edge; follow until the tap.

```
static List<int[]> waterConnections(int n, int[][] pipes) {
    int[] next = new int[n + 1], diameter = new int[n + 1], indegree = new int[n + 1];
    for (int[] p : pipes) { next[p[0]] = p[1]; diameter[p[0]] = p[2]; indegree[p[1]]++; }
    List<int[]> result = new ArrayList<>();
    for (int tank = 1; tank <= n; tank++) if (indegree[tank] == 0 && next[tank] != 0) {
        int house = tank, minDiameter = Integer.MAX_VALUE;
        while (next[house] != 0) {
            minDiameter = Math.min(minDiameter, diameter[house]);
            house = next[house];
        }
        result.add(new int[]{tank, house, minDiameter});
    }
    return result;
}
```

Tips and edge cases

This compact array representation relies on the problem guarantee of at most one outgoing and incoming pipe per house and no cycles in valid networks. For arbitrary directed graphs, use adjacency lists and explicit cycle detection.

GeeksforGeeks: [Open source problem / topic](#)

22. Check whether brackets are balanced

Problem: Given a string containing brackets, return true if every opening bracket has the matching closing type and all brackets close in the correct order. Non-bracket characters are ignored.

Pattern: Stack; the most recently opened bracket must be the next one closed.

1. Brute force: repeatedly remove adjacent matched pairs

$O(n^2)$ time, $O(n)$ space

Delete (), [], or {} pairs until no more are found; the remainder is balanced only if empty.

```
static boolean balancedBrute(String s) {
    StringBuilder x = new StringBuilder(s);
    boolean changed = true;
    while (changed) {
        changed = false;
        for (int i = 0; i + 1 < x.length(); i++) {
            String pair = x.substring(i, i + 2);
            if (pair.equals("(") || pair.equals("[") || pair.equals("{}")) {
                x.delete(i, i + 2); changed = true; break;
            }
        }
    }
    for (int i = 0; i < x.length(); i++)
        if ("()[]{}".indexOf(x.charAt(i)) >= 0) return false;
    return true;
}
```

2. Better: count each bracket type

$O(n)$ time, $O(1)$ space; only works for one bracket type

For parentheses only, track balance and reject a negative prefix or nonzero final count.

```
static boolean balancedParenOnly(String s) {
    int balance = 0;
    for (char c : s.toCharArray()) {
        if (c == '(') balance++;
        else if (c == ')' && --balance < 0) return false;
    }
    return balance == 0;
}
```

3. Best: stack opening bracket types

$O(n)$ time, $O(n)$ space

Push openers; each closer must match the stack top, and the stack must be empty at the end.

```
static boolean isBalanced(String s) {
    Deque<Character> st = new ArrayDeque<>();
    for (char c : s.toCharArray()) {
        if (c == '(' || c == '[' || c == '{') st.push(c);
        else if (c == ')' || c == ']' || c == '}') {
            if (st.isEmpty()) return false;
            char open = st.pop();
            if ((c == ')' && open != '(') || (c == ']' && open != '[') || (c == '}' && open != '{')) return false;
        }
    }
    return st.isEmpty();
}
```

Tips and edge cases

A counter is enough only when there is one bracket type. With multiple bracket types, nesting order matters. Test early closing, mismatched type, leftover opener, and ordinary text.

GeeksforGeeks: [Open source problem / topic](#)

LeetCode matches or related practice: [Valid Parentheses](#) • [Longest Valid Parentheses](#)

23. Evaluate an arithmetic expression using stacks

Problem: Evaluate an infix expression containing nonnegative integer literals, parentheses, and binary $+$, $-$, $*$, $/$. Division truncates toward zero. Whitespace is allowed; unary operators are excluded to keep this starter version focused.

Pattern: Operator precedence and parentheses; either parse by precedence recursively or use operator/value stacks.

1. Brute force: recursively split at the lowest-precedence operator

$O(n^2)$ time, $O(n)$ recursion space

Strip outer parentheses, find a top-level $+$ or $-$ from right to left; otherwise find $*$ or $/$. Split and recursively evaluate both sides.

```
static long evalRecursive(String s) {
    s = stripOuter(s.trim());
    int depth = 0;
    for (int i = s.length() - 1; i >= 0; i--) {
        char c = s.charAt(i);
        if (c == ')') depth++;
        else if (c == '(') depth--;
        else if (depth == 0 && i > 0 && (c == '+' || c == '-')) {
            long a = evalRecursive(s.substring(0, i)), b = evalRecursive(s.substring(i + 1));
            return c == '+' ? a + b : a - b;
        }
    }
    depth = 0;
    for (int i = s.length() - 1; i >= 0; i--) {
        char c = s.charAt(i);
        if (c == ')') depth++;
        else if (c == '(') depth--;
        else if (depth == 0 && (c == '*' || c == '/')) {
            long a = evalRecursive(s.substring(0, i)), b = evalRecursive(s.substring(i + 1));
            return c == '*' ? a * b : a / b;
        }
    }
    if (s.startsWith("(")) return evalRecursive(s.substring(1, s.length() - 1));
    return Long.parseLong(s);
}

static String stripOuter(String s) {
    while (s.startsWith("(") && s.endsWith("))")) {
        int depth = 0; boolean wrapsAll = true;
        for (int i = 0; i < s.length(); i++) {
            if (s.charAt(i) == '(') depth++;
            if (s.charAt(i) == ')') depth--;
            if (depth == 0 && i < s.length() - 1) { wrapsAll = false; break; }
        }
        if (!wrapsAll) break;
        s = s.substring(1, s.length() - 1);
    }
    return s;
}
```

2. Better: operator/value stacks with precedence

O(n) time, O(n) space

Push operators and values; reduce higher or equal precedence before pushing a new operator, and reduce to the opening parenthesis at a close.

```
static long evalStacks(String s) {
    Deque<Long> values = new ArrayDeque<>();
    Deque<Character> ops = new ArrayDeque<>();
    for (int i = 0; i < s.length(); i++) {
        char c = s.charAt(i);
        if (Character.isWhitespace(c)) { i++; continue; }
        if (Character.isDigit(c)) {
            long v = 0;
            while (i < s.length() && Character.isDigit(s.charAt(i))) v = v * 10 + (s.charAt(i++) - '0');
            values.push(v); continue;
        }
        if (c == '(') ops.push(c);
        else if (c == ')') {
            while (!ops.isEmpty() && ops.peek() != '(') reduce(values, ops.pop());
            ops.pop();
        } else {
            while (!ops.isEmpty() && ops.peek() != '(' && prec(ops.peek()) >= prec(c)) reduce(values, ops.pop());
            ops.push(c);
        }
        i++;
    }
    while (!ops.isEmpty()) reduce(values, ops.pop());
    return values.pop();
}

static int prec(char op) { return (op == '+' || op == '-') ? 1 : 2; }
static void reduce(Deque<Long> v, char op) {
    long b = v.pop(), a = v.pop();
    if (op == '+') v.push(a + b); else if (op == '-') v.push(a - b);
    else if (op == '*') v.push(a * b); else v.push(a / b);
}
```

3. Best for learning: recursive descent parser

O(n) time, O(n) call stack

Parse an expression as terms joined by +/−, a term as factors joined by */ , and a factor as a number or parenthesized expression.

```
static class Parser {
    final String s; int i = 0;
    Parser(String s) { this.s = s; }
    long expression() {
        long value = term();
        while (true) {
            skip(); if (i == s.length() || s.charAt(i) == ')') return value;
            char op = s.charAt(i++); long rhs = term();
            value = op == '+' ? value + rhs : value - rhs;
        }
    }
    long term() {
        long value = factor();
        while (true) {
            skip(); if (i == s.length() || s.charAt(i) == ') || s.charAt(i) == '+' || s.charAt(i) == '-') return value;
            char op = s.charAt(i++); long rhs = factor();
            value = op == '*' ? value * rhs : value / rhs;
        }
    }
    long factor() {
        skip();
        if (s.charAt(i) == '(') { i++; long v = expression(); skip(); i++; return v; }
        long v = 0;
        while (i < s.length() && Character.isDigit(s.charAt(i))) v = v * 10 + (s.charAt(i++) - '0');
        return v;
    }
    void skip() { while (i < s.length() && Character.isWhitespace(s.charAt(i))) i++; }
}

static long evaluate(String s) { return new Parser(s).expression(); }
```


Tips and edge cases

The code assumes valid expressions, binary operators, and no division by zero. Integer division truncates toward zero. For the recursive split version, substring creation causes extra copying; the one-pass parser is preferable for long inputs.

GeeksforGeeks: [Open source problem / topic](#)

LeetCode matches or related practice: [Basic Calculator](#) • [Basic Calculator II](#)

24. Next Greater Element to the right

Problem: For each array element, return the first value to its right that is strictly greater. Return -1 when no greater value exists.

Pattern: Monotonic decreasing stack of unresolved indices; a new value resolves all smaller values on top.

1. Brute force: scan to the right

$O(n^2)$ time, $O(n)$ output space

For each position, stop at the first greater value.

```
static int[] nextGreaterBrute(int[] a) {
    int[] ans = new int[a.length];
    for (int i = 0; i < a.length; i++) {
        ans[i] = -1;
        for (int j = i + 1; j < a.length; j++) if (a[j] > a[i]) { ans[i] = a[j]; break; }
    }
    return ans;
}
```

2. Better: jump through previously computed answers

$O(n^2)$ worst case, $O(n)$ output space

When a candidate is too small, jump to its next-greater index to skip values that cannot answer the current query.

```
static int[] nextGreaterJump(int[] a) {
    int n = a.length; int[] ans = new int[n], next = new int[n];
    Arrays.fill(ans, -1); Arrays.fill(next, -1);
    for (int i = n - 2; i >= 0; i--) {
        int j = i + 1;
        while (j != -1 && a[j] <= a[i]) j = next[j];
        next[i] = j;
        if (j != -1) ans[i] = a[j];
    }
    return ans;
}
```

3. Best: monotonic stack

$O(n)$ time, $O(n)$ space

Scan left to right. Pop indices whose values are smaller than the current value and assign their answer.

```
static int[] nextGreater(int[] a) {
    int[] ans = new int[a.length]; Arrays.fill(ans, -1);
    Deque<Integer> st = new ArrayDeque<>();
    for (int i = 0; i < a.length; i++) {
        while (!st.isEmpty() && a[st.peek()] < a[i]) ans[st.pop()] = a[i];
        st.push(i);
    }
    return ans;
}
```

Tips and edge cases

Use indices in the stack if answers need positions or if values repeat. Strictly greater means equal values do not resolve each other. Circular next-greater variants scan the array twice.

GeeksforGeeks: [Open source problem / topic](#)

LeetCode matches or related practice: [Next Greater Element I](#) • [Next Greater Element II](#) • [Daily Temperatures](#)

25. Implement a stack using an array

Problem: Implement push, pop, peek, and isEmpty using an array. Define what pop/peek do on an empty stack; this workbook throws NoSuchElementException.

Pattern: Stack is LIFO; store the next free index as the current size so push/pop update one end.

1. Brute force: keep the top at index 0

Push/pop $O(n)$, peek $O(1)$, $O(n)$ space

Insert and remove at the front of an array-backed list, shifting elements each time.

```
static class ShiftingStack {
    ArrayList<Integer> data = new ArrayList<>();
    void push(int x) { data.add(0, x); }
    int pop() {
        if (data.isEmpty()) throw new NoSuchElementException();
        return data.remove(0);
    }
    int peek() {
        if (data.isEmpty()) throw new NoSuchElementException();
        return data.get(0);
    }
    boolean isEmpty() { return data.isEmpty(); }
}
```

2. Better: use the end of a dynamic list

Push/pop amortized $O(1)$, peek $O(1)$, $O(n)$ space

Append the top to the end; removing the last element does not shift the other values.

```
static class ListStack {
    ArrayList<Integer> data = new ArrayList<>();
    void push(int x) { data.add(x); }
    int pop() {
        if (data.isEmpty()) throw new NoSuchElementException();
        return data.remove(data.size() - 1);
    }
    int peek() {
        if (data.isEmpty()) throw new NoSuchElementException();
        return data.get(data.size() - 1);
    }
    boolean isEmpty() { return data.isEmpty(); }
}
```

3. Best for the array exercise: custom resizable array

Push/pop amortized $O(1)$, peek $O(1)$, $O(n)$ space

Track size as the next free slot and double capacity when full.

```
static class ArrayStack {
    private int[] data = new int[4];
    private int size = 0;
    void push(int x) {
        if (size == data.length) data = Arrays.copyOf(data, data.length * 2);
        data[size++] = x;
    }
    int pop() {
        if (size == 0) throw new NoSuchElementException();
        return data[--size];
    }
    int peek() {
        if (size == 0) throw new NoSuchElementException();
        return data[size - 1];
    }
    boolean isEmpty() { return size == 0; }
    int size() { return size; }
}
```

Tips and edge cases

Keep size equal to the number of stored elements; valid indexes are 0 through size-1. A fixed-capacity assignment may require an overflow condition instead of resizing. Java also provides ArrayDeque for production stack-like use.

GeeksforGeeks: [Open source problem / topic](#)

LeetCode matches or related practice: [Implement Stack Using Queues](#) • [Min Stack](#)

26. Implement a queue using a linked list

Problem: Implement enqueue, dequeue, peek, and isEmpty for a FIFO queue backed by linked nodes. Enqueue at the tail and dequeue at the head.

Pattern: Maintain head and tail pointers; each operation changes one endpoint in constant time.

1. Brute force: array list with front removal

Enqueue $O(1)$ amortized, dequeue $O(n)$, $O(n)$ space

Add at the back and remove index zero; removal shifts all remaining values.

```
static class ListQueue {
    ArrayList<Integer> a = new ArrayList<>();
    void enqueue(int x) { a.add(x); }
    int dequeue() {
        if (a.isEmpty()) throw new NoSuchElementException();
        return a.remove(0);
    }
    int peek() {
        if (a.isEmpty()) throw new NoSuchElementException();
        return a.get(0);
    }
}
```

2. Better: use two stacks

Amortized $O(1)$ enqueue/dequeue, $O(n)$ space

Push new values onto an input stack. When the output stack is empty, transfer all values to reverse their order.

```
static class TwoStackQueue {
    Deque<Integer> in = new ArrayDeque<>(), out = new ArrayDeque<>();
    void enqueue(int x) { in.push(x); }
    private void prepare() {
        if (out.isEmpty()) while (!in.isEmpty()) out.push(in.pop());
    }
    int dequeue() { prepare(); if (out.isEmpty()) throw new NoSuchElementException(); return out.pop(); }
    int peek() { prepare(); if (out.isEmpty()) throw new NoSuchElementException(); return out.peek(); }
    boolean isEmpty() { return in.isEmpty() && out.isEmpty(); }
}
```

3. Best for a linked-list queue: head/tail nodes

Enqueue/dequeue/peek $O(1)$, $O(n)$ space

Append at tail, remove from head, and set both pointers to null when the last node is removed.

```
static class LinkedQueue {
    static class Node { int value; Node next; Node(int v) { value = v; } }
    private Node head, tail;
    private int size;
    void enqueue(int x) {
        Node node = new Node(x);
        if (tail == null) head = tail = node;
        else { tail.next = node; tail = node; }
        size++;
    }
    int dequeue() {
        if (head == null) throw new NoSuchElementException();
        int value = head.value; head = head.next;
        if (head == null) tail = null;
        size--; return value;
    }
    int peek() { if (head == null) throw new NoSuchElementException(); return head.value; }
    boolean isEmpty() { return head == null; }
    int size() { return size; }
}
```

Tips and edge cases

A common bug is forgetting `tail=null` when dequeue removes the last node; the next enqueue would otherwise attach to a stale tail. Test empty, one-item, and repeated enqueue/dequeue cases.

GeeksforGeeks: [Open source problem / topic](#)

LeetCode matches or related practice: [Implement Queue Using Stacks](#) • [Number Of Recent Calls](#)

27. Implement a queue using an array

Problem: Implement FIFO enqueue and dequeue operations using an array. A simple shifting queue is easy to write but slow; a circular array reuses freed slots.

Pattern: Array queue; head and tail indexes wrap with modulo capacity.

1. Brute force: shift elements after dequeue

Enqueue $O(1)$ amortized, dequeue $O(n)$, $O(n)$ space

Store elements in order and shift all remaining elements left when removing the front.

```
static class ShiftingArrayQueue {
    int[] a = new int[16]; int size = 0;
    void enqueue(int x) {
        if (size == a.length) throw new IllegalStateException("full");
        a[size++] = x;
    }
    int dequeue() {
        if (size == 0) throw new NoSuchElementException();
        int x = a[0]; System.arraycopy(a, 1, a, 0, --size); return x;
    }
}
```

2. Better: fixed circular array

Enqueue/dequeue $O(1)$, $O(C)$ space

Use head and size; compute the insertion slot as $(\text{head} + \text{size}) \bmod \text{capacity}$.

```
static class FixedArrayQueue {
    int[] a; int head = 0, size = 0;
    FixedArrayQueue(int capacity) { a = new int[capacity]; }
    boolean offer(int x) {
        if (size == a.length) return false;
        a[(head + size) % a.length] = x; size++; return true;
    }
    int poll() {
        if (size == 0) throw new NoSuchElementException();
        int x = a[head]; head = (head + 1) % a.length; size--; return x;
    }
}
```

3. Best for a general-purpose array queue: resizing ring buffer

Amortized $O(1)$ enqueue/dequeue, $O(n)$ space

When full, copy logical queue order into a larger array and reset head to zero.

```
static class ResizingArrayQueue {
    int[] a = new int[4]; int head = 0, size = 0;
    void enqueue(int x) {
        if (size == a.length) resize(a.length * 2);
        a[(head + size) % a.length] = x; size++;
    }
    int dequeue() {
        if (size == 0) throw new NoSuchElementException();
        int x = a[head]; head = (head + 1) % a.length; size--;
        if (a.length > 4 && size <= a.length / 4) resize(a.length / 2);
        return x;
    }
    void resize(int cap) {
        int[] b = new int[cap];
        for (int i = 0; i < size; i++) b[i] = a[(head + i) % a.length];
        a = b; head = 0;
    }
}
```

Tips and edge cases

A circular queue must distinguish empty from full; tracking size makes that explicit. Handle zero capacity before using modulo. Resizing copies in logical FIFO order, not physical array order.

GeeksforGeeks: [Open source problem / topic](#)

LeetCode matches or related practice: [Implement Queue Using Stacks](#) • [Design Circular Queue](#)

28. Design a circular queue

Problem: Create a fixed-capacity circular queue with `enqueue`, `dequeue`, `front`, `rear`, `isEmpty`, and `isFull`. Return `false` or `-1` as the specified interface requires when an operation cannot be performed.

Pattern: Ring buffer; head and size are enough to locate front and rear without shifting values.

1. Brute force: shift after every deletion

Enqueue $O(1)$, dequeue $O(n)$, $O(k)$ space

This linear queue works until the rear reaches the end; shifting recovers space but defeats circular behavior.

```
static class LinearQueue {
    int[] a; int size = 0;
    LinearQueue(int k) { a = new int[k]; }
    boolean enqueue(int x) { if (size == a.length) return false; a[size++] = x; return true; }
    boolean dequeue() {
        if (size == 0) return false;
        System.arraycopy(a, 1, a, 0, --size); return true;
    }
    int front() { return size == 0 ? -1 : a[0]; }
    int rear() { return size == 0 ? -1 : a[size - 1]; }
}
```

2. Better: fixed circular array with head and size

All queue operations $O(1)$, $O(k)$ space

Wrap the insertion and removal indexes with modulo `k`. A `size` field distinguishes full from empty.

```
static class CircularQueue {
    int[] a; int head = 0, size = 0;
    CircularQueue(int k) { a = new int[k]; }
    boolean enqueue(int x) {
        if (size == a.length) return false;
        a[(head + size) % a.length] = x; size++; return true;
    }
    boolean dequeue() {
        if (size == 0) return false;
        head = (head + 1) % a.length; size--; return true;
    }
    int front() { return size == 0 ? -1 : a[head]; }
    int rear() { return size == 0 ? -1 : a[(head + size - 1) % a.length]; }
    boolean isEmpty() { return size == 0; }
    boolean isFull() { return size == a.length; }
}
```

3. Best: sentinel-index ring buffer

All queue operations $O(1)$, $O(k+1)$ space

Store front and rear indexes with one unused slot; equal indexes mean empty and `next(rear) == front` means full.

```
static class SentinelCircularQueue {
    int[] a; int front = 0, rear = 0;
    SentinelCircularQueue(int k) { a = new int[k + 1]; }
    boolean enqueue(int x) {
        int next = (rear + 1) % a.length;
        if (next == front) return false;
        a[rear] = x; rear = next; return true;
    }
    boolean dequeue() { if (front == rear) return false; front = (front + 1) % a.length; return true; }
    int front() { return front == rear ? -1 : a[front]; }
    int rear() { return front == rear ? -1 : a[(rear - 1 + a.length) % a.length]; }
    boolean isEmpty() { return front == rear; }
    boolean isFull() { return (rear + 1) % a.length == front; }
}
```

Tips and edge cases

Test capacity 1, fill then reject one more insertion, remove then insert after wraparound, and empty front/rear. A fixed queue of capacity `k` using an unused sentinel slot needs an internal array of length `k+1`.

GeeksforGeeks: [Open source problem / topic](#)

LeetCode matches or related practice: [Design Circular Queue](#)

29. Reverse the first k elements of a queue

Problem: Given a queue and integer k, reverse only its first k elements and leave the remaining elements in their original relative order. Assume $0 \leq k \leq \text{queue size}$.

Pattern: Queue plus LIFO auxiliary storage; remove the first k, reverse them, append them, then rotate the untouched suffix behind them.

1. Brute force: rotate and insert by position

$O(nk)$ time, $O(k)$ extra space

Copy the first k values into a list, reverse that list, rebuild the queue, then append the suffix.

```
static void reverseFirstKBrute(Queue<Integer> q, int k) {
    if (k < 0 || k > q.size()) throw new IllegalArgumentException();
    List<Integer> first = new ArrayList<>();
    for (int i = 0; i < k; i++) first.add(q.remove());
    Collections.reverse(first);
    Queue<Integer> rest = new ArrayDeque<>(q); q.clear();
    q.addAll(first); q.addAll(rest);
}
```

2. Better: reverse using a stack

$O(n)$ time, $O(k)$ extra space

Push the first k dequeued values onto a stack, pop them back into the queue, then rotate the n-k suffix.

```
static void reverseFirstKStack(Queue<Integer> q, int k) {
    if (k < 0 || k > q.size()) throw new IllegalArgumentException();
    int n = q.size(); Deque<Integer> st = new ArrayDeque<>();
    for (int i = 0; i < k; i++) st.push(q.remove());
    while (!st.isEmpty()) q.add(st.pop());
    for (int i = 0; i < n - k; i++) q.add(q.remove());
}
```

3. Best for large k: iterative stack with a deque interface

$O(n)$ time, $O(k)$ extra space

Use the same three phases, but record the original queue size once so rotating the suffix does not depend on a changing size.

```
static void reverseFirstKDeque(Queue<Integer> q, int k) {
    int n = q.size();
    if (k < 0 || k > n) throw new IllegalArgumentException();
    Deque<Integer> st = new ArrayDeque<>();
    for (int i = 0; i < k; i++) st.push(q.removeFirst());
    while (!st.isEmpty()) q.addLast(st.pop());
    for (int i = 0; i < n - k; i++) q.addLast(q.removeFirst());
}
```

Tips and edge cases

Record the initial queue size before changing it. When $k=0$ or $k=n$, the same algorithm still works. Preserve the suffix order by rotating it exactly $n-k$ times.

GeeksforGeeks: [Open source problem / topic](#)

LeetCode matches or related practice: [Reveal Cards In Increasing Order](#) (related queue ordering; different task)